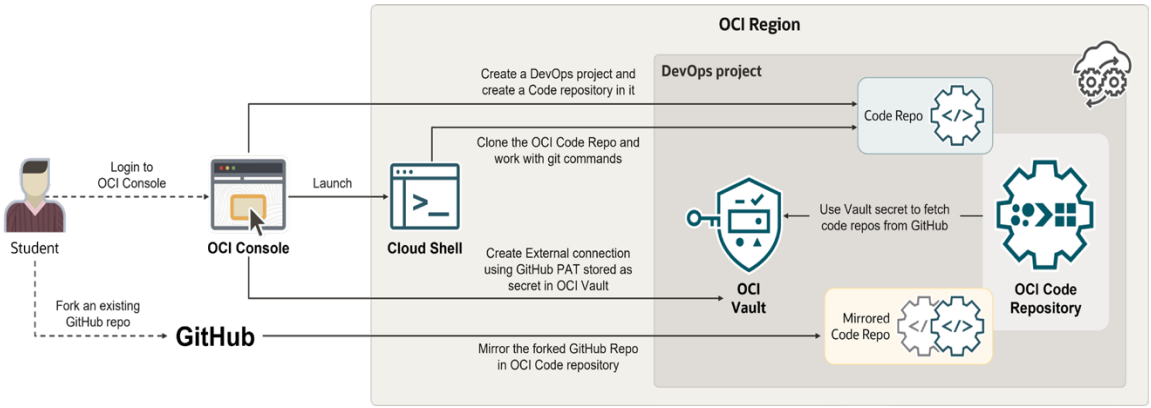
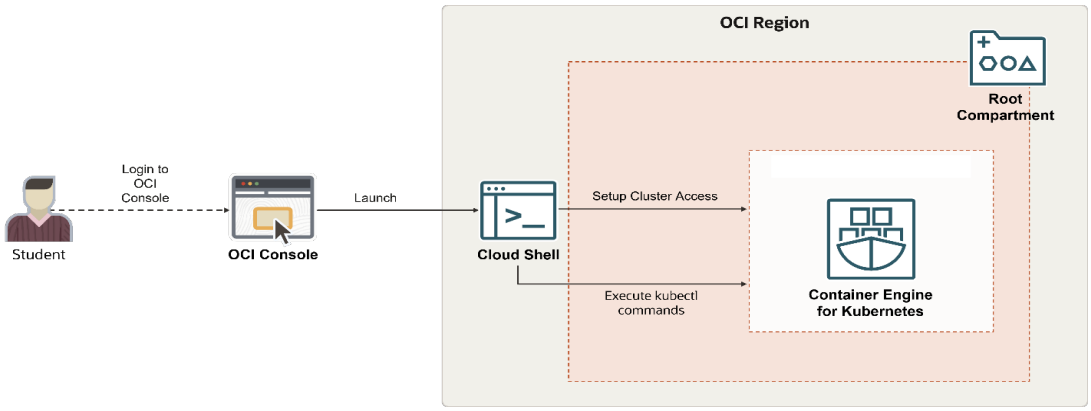
Lab 5. Deploy a load-balanced Web App on an OKE using Kubectl

Overview

In the first part of the lab, you'll create a sample repository. You need to set up access to your Kubernetes cluster to deploy your application. The `kubectl` command-line client is a versatile way to interact with a Kubernetes cluster, including managing multiple clusters.





In this lab, you'll:

a. Create a DevOps project.
b. Create an OCI Code Repository in your DevOps project.
c. Clone OCI Code Repository to your instance.
d. Perform basic Git operations on the Code Repository.
e. Set up the `kubeconfig` file in the cluster for an existing OKE
f. Run `kubectl` commands against Kubernetes cluster.

Assumptions

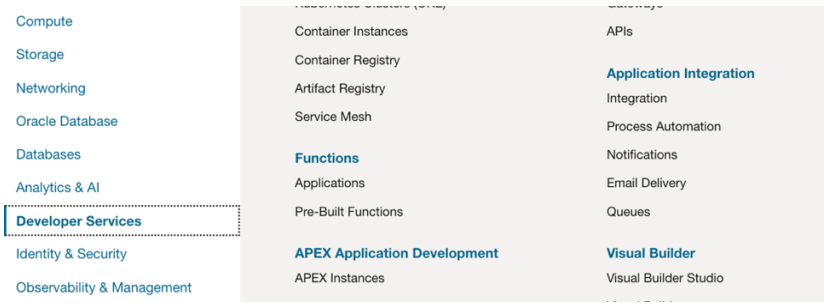
- This lab assumes you're working in the Ashburn region. The resource naming convention

(`iad`) used in this lab is according to Ashburn.

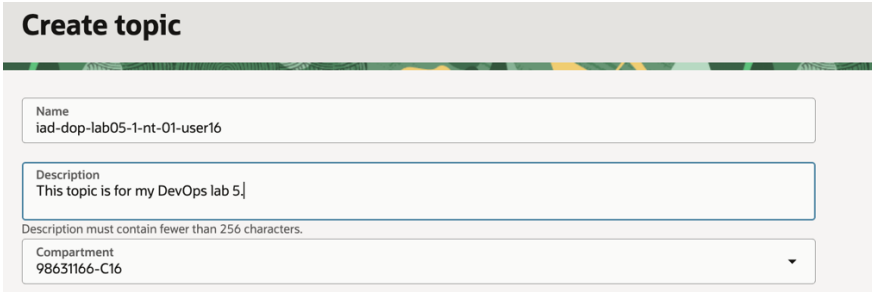
- You will replace the *<UserID>* placeholder with your actual user id..
- You are signed in to your Oracle Cloud Infrastructure (OCI) account using your credentials.
- You have the Access Token created in the earlier **Lab 4, " AIOPS-LAB04-AT-01"**
- You will replace the *<userID>* placeholder with your user ID (e.g. `user16`).

Sequence 1. Create a DevOps Project

1. In the Console, open the navigation menu and click **Developer Services**. Under **Application Integration**, click **Notifications**.

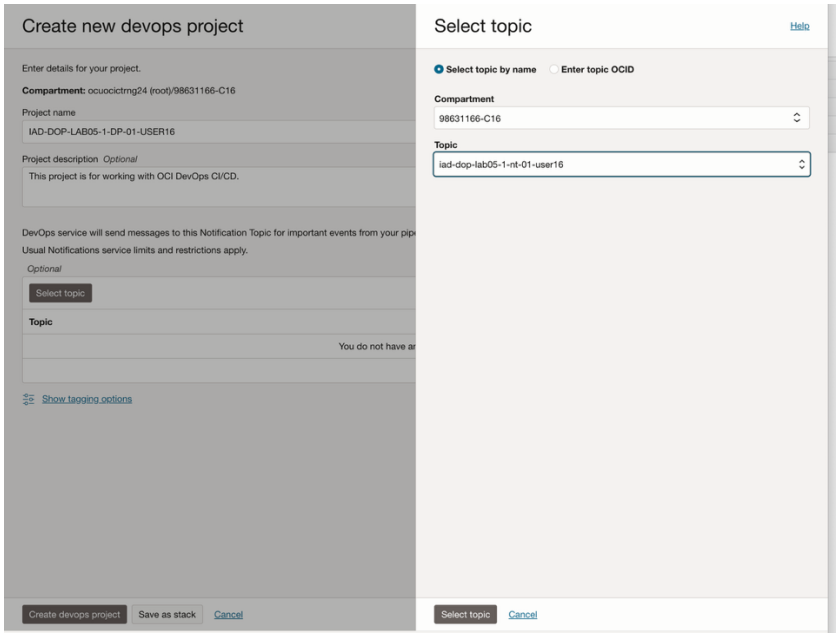


2. Select your *<Compartment Name>* from applied fileters
3. Click **Topics** under the notification in the **left menu.**
4. Click **Create Topic** at the top of the topic list.
5. In the **Create Topic** page, configure your topic and click **Create**.
   - **Name**: `iad-dop-lab05-1-nt-01-`*<userID>*.
   - **Description**: `This topic is for my DevOps lab 5.`



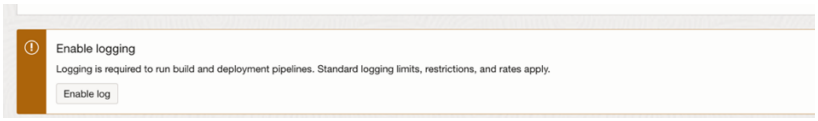
6. From **Developer Services**, under **DevOps**, click **Projects**.
7. **Select** your *<Compartment Name>* from List scope on the left menu.
8. Click **Create devops project**.
   - **Name**: `IAD-DOP-LAB05-1-DP-01-`*<userID>*
   - **Description**: `This project is for working with OCI DevOps CI/CD.`
   - To set up project notifications, click **Select Topic**.
     - In the Select topic window. Select the option **"Select topic by name"**
     - In the **compartment** field, select your *<Compartment Name>*

- In the **Topic** field, select the topic `iad-dop-lab05-1-nt-01-<userID>`.
- Click **Select Topic** at the bottom.

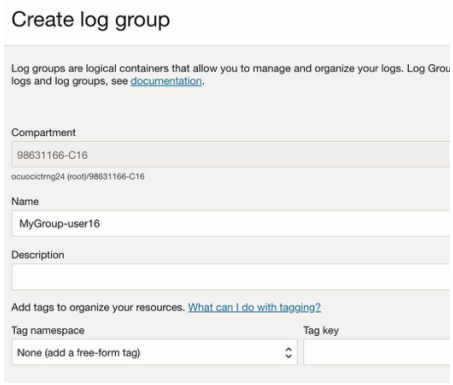


- Click **Create devops project**.

9. On the page of your newly created project, click **Enable Log** which takes you to the log management page.

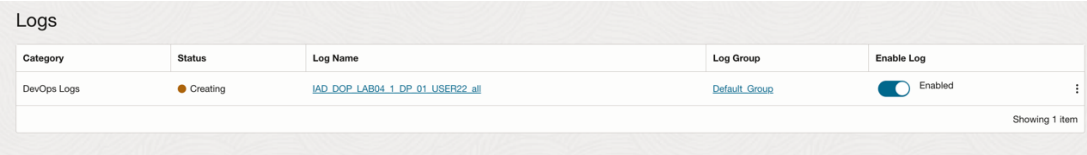


10. In the **Logs** table, toggle to **enable** the log. This will pop-up to Enable Log window. You need a Log Group also. Click on ***create log group*** to create it.



11. Leave all the options as default and click **Enable Log** at the bottom.

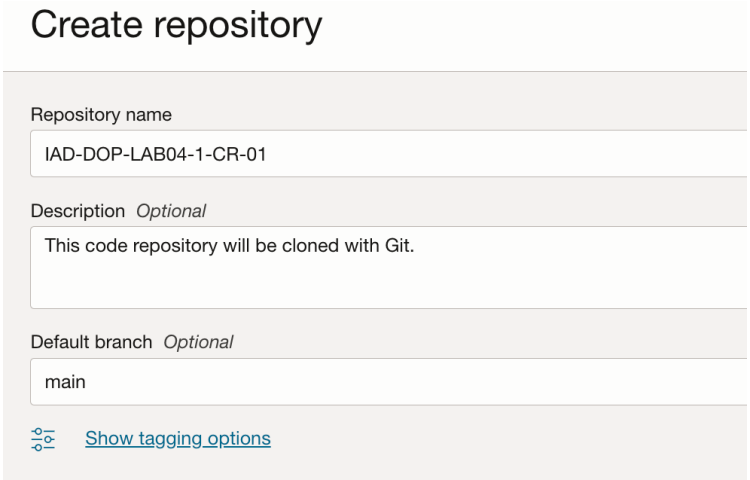
12. The logs will go through the **Creating** state to the **Active** state.



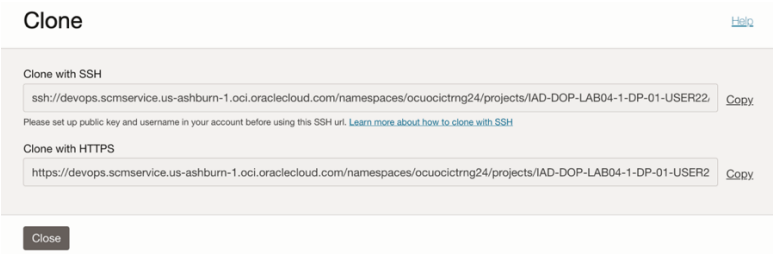
Sequence 2. Create an OCI Code Repository and Clone it

You'll learn to create a code repository inside your DevOps project, which is very similar to your Git repository.

1. Click **Code Repositories** on the left menu of your project page.
2. Click **Create Repository**. Enter the following details:
   - **Repository name**: `IAD-DOP-LAB05-1-CR-01`
   - **Description**: `This code repository will be cloned with Git.`
   - **Default branch**: `main`



3. Click **Create Repository**.
4. Click **Clone** in the Code Repository details page.
5. In the **Clone** window, to the right of the **Clone with HTTPS** field, click **Copy** to get the path to access the repository using Git. Save this information in a notepad.



6. In the Terminal of your Jupyter Lab and paste the URL to clone the public repository.

   a. Clone by copy-pasting the URL.

      ```
      git clone <paste the HTTPS URL copied in the Clone page.>
      ```

   b. You must provide your username: `<tenancy-namespace>/<username>`.

      For example, `ocuocictrng24/98631166-lab.user16`

   c. Your password is your auth token.

7. You can now add the files from your existing `docker-helloworld-demo` (**Lab 4, Sequence 2. Step 4)** directory to the **IAD-DOP-LAB04-1-CR-01** directory you just cloned.

   ```
   cd ~/docker-helloworld-demo
   cp * ~/IAD-DOP-LAB05-1-CR-01
   ```

8. Verify the files copied.

   ```
   cd ~/IAD-DOP-LAB05-1-CR-01 && ls
   ```

9. Every time you make changes to your files and save it, it will not automatically update the OCI Code Repository (**IAD-DOP-LAB05-1-CR-01**) within the DevOps Project (**IAD-DOP-LAB05-1-DP-01-<*userID*>**). All the changes you made in the file are updated only in your local repository. To update the changes to the main branch in OCI Code Repository within the DevOps Project run the following commands:

   ```
   git add .
   git config --global user.email "ramnsangwan@gmail.com"
   git config --global user.name "Ram N Sangwan"
   git commit -m "first push into OCI Code Repository"
   git push -u -f origin main
   ```

- When it prompts for your username: `<tenancy-namespace>/<username>`. For example, `ocuocictrng24/98631166-lab.user16`.

  Your password is the auth token

10. In the OCI Console, go to your DevOps project and then to the IAD-DOP-LAB05-1-CR-01 code repository you created. Click **Files** in the left menu and notice all the files are available in the code repository.

Sequence 3. Set Up the kubeconfig File

1. Launch the Jupyter Lab and Switch to root user and uninstall the K3S

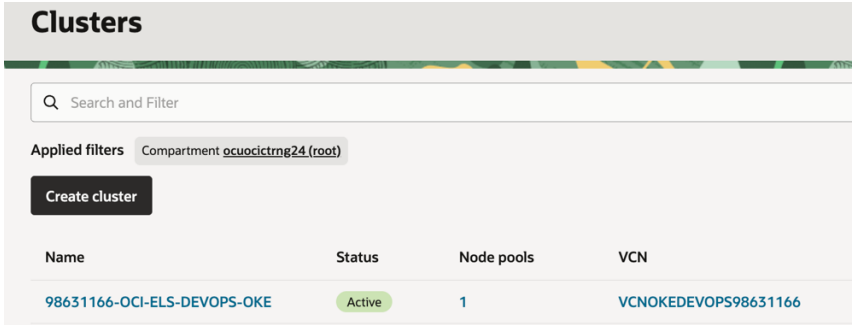
```
sudo su -
[root@aiops-labs ~]# k3s-uninstall.sh
[root@aiops-labs ~]# rm -rf .kube
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"

sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl

kubectl version —client
```

2. Setup oci config file for **root** user. For details refer to ***Lab 1a OCI CLI Setup. Screen shot are for your reference.***

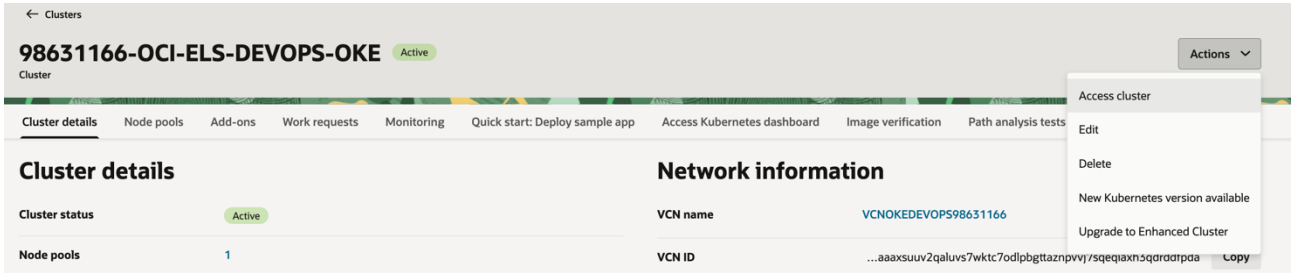
```
[root@aiops-labs ~]# oci setup config
    This command provides a walkthrough of creating a valid CLI config file.

    The following links explain where to find the information required by this
    script:

    User API Signing Key, OCID and Tenancy OCID:

        https://docs.cloud.oracle.com/Content/API/Concepts/apisigningkey.htm#Other

    Region:

        https://docs.cloud.oracle.com/Content/General/Concepts/regions.htm

    General config documentation:

        https://docs.cloud.oracle.com/Content/API/Concepts/sdkconfig.htm


Enter a location for your config [/root/.oci/config]:
Enter a user OCID: ocid1.user.oc1..aaaaaaaaxfapfyv5svxrpfonh3r3ydeifw6hfshdoxtkqy2wobevaiksrkwa
Enter a tenancy OCID: ocid1.tenancy.oc1..aaaaaaaa753kybqghsyuluysjgx66fziyjefpst5pgpbghwpr3bgbu7glwvq
Enter a region by index or name(e.g.
```

```
81: us-saltlake-2, 82: us-sanjose-1, 83: us-somerset-1, 84: us-thames-1): us-ashburn-1
Do you want to generate a new API Signing RSA key pair? (If you decline you will be asked to supply the path to an existing key.) [Y/n]:
Enter a directory for your keys to be created [/root/.oci]:
Enter a name for your key [oci_api_key]:
Public key written to: /root/.oci/oci_api_key_public.pem
Enter a passphrase for your private key ("N/A" for no passphrase):
Repeat for confirmation:
Private key written to: /root/.oci/oci_api_key.pem
Fingerprint: 90:5f:da:19:44:59:3a:b7:f5:88:61:60:28:29:57:34
Config written to /root/.oci/config
```

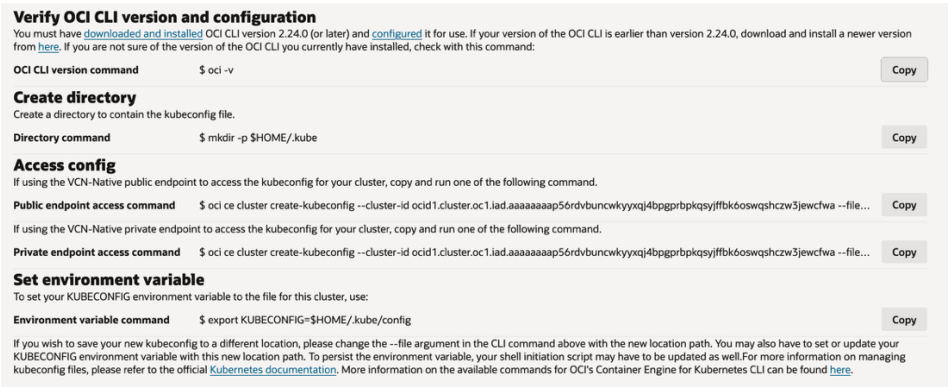
3. Login to Oracle Cloud Web Console and select the **US-East (Ashburn)** Region from Top right corner of web console.
4. In the Console, open the navigation menu and click Developer Services. Under Containers and Artifacts, click **Kubernetes Clusters (OKE)**.
5. Choose root compartment from List Scope on the left menu.
6. On the Cluster List page, click **<Event-ID>-OCI-ELS-DEVOPS-OKE** cluster you want to access using `kubectl`. The Cluster page shows the details of the cluster.



7. Click on the Actions drop down and then **Access Cluster** to display the **Access Your Cluster** window.



8. Click **Local Access** and copy the command to access the `kubeconfig` for your cluster via the VCN-Native "**Public endpoint access command**" endpoint and paste it on notepad.



9. Now run the copied command from the terminal of Jupyter Lab (as root user).
10. On successful execution, it will return a new config written to `kubeconfig` file.

```
[root@aiops-labs ~]# oci ce cluster create-kubeconfig --cluster-id ocid1.cluster.oc1.iad.aaaaaaa
 2.0.0  --kube-endpoint PUBLIC_ENDPOINT
New config written to the Kubeconfig file /root/.kube/config
[root@aiops-labs ~]#
```

For example,

```
$ oci ce cluster create-kubeconfig \
--cluster-id ocid1.cluster.oc1.iad.xxxxxaaxxxxx \
--file $HOME/.kube/config \
--region us- ashburn-1 --token-version 2.0.0 \
--kube-endpoint PUBLIC_ENDPOINT
```

Note: This is just a representation of the command. Do not use this command to connect with the cluster that's created for this lab.

Sequence 4. Run kubectl Commands Against Kubernetes Clusters

Having set up the `kubeconfig` file, you can start using `kubectl` to access the cluster by creating a sample deployment in OKE cluster. To save time, commands are given in a text file also.

1. Verify that kubectl can connect to the cluster.

```
kubectl get nodes
```

2. Create namespace in your Kubernetes cluster to manage your resources.

```
kubectl create ns ns-<userID>
kubectl create ns ns-user16
```

```
[root@aiops-labs ~]# kubectl get nodes
NAME          STATUS   ROLES   AGE     VERSION
10.0.10.127   Ready    node    2d10h   v1.33.1
10.0.10.74    Ready    node    2d10h   v1.33.1
10.0.10.9     Ready    node    2d10h   v1.33.1
[root@aiops-labs ~]# kubectl create ns ns-user16
namespace/ns-user16 created
[root@aiops-labs ~]#
```

3. View the cluster information.

```
kubectl cluster-info
```

4. Create a deployment in OKE cluster.

```
kubectl create deployment deploy-<userID> \
--image=iad.ocir.io/ocuocictrng5/httpd:latest -n ns-<userID>
```

Don't Change this.E.g

```
kubectl create deployment deploy-user16 \
--image=iad.ocir.io/ocuocictrng5/httpd:latest -n ns-user16
```

5. Expose your deployment using service of type load balancer by using the following command.

```
kubectl expose deployment deploy-user16 \
--type=LoadBalancer \
--name=svc-user16 --port=80 \
--target-port=80 -n ns-user16
```

```
[root@aiops-labs ~]# kubectl create deployment deploy-user16 \
--image=iad.ocir.io/ocuocictrng5/httpd:latest -n ns-user16
deployment.apps/deploy-user16 created
[root@aiops-labs ~]# kubectl expose deployment deploy-user16 \
--type=LoadBalancer \
--name=svc-user16 --port=80 \
--target-port=80 -n ns-user16
service/svc-user16 exposed
[root@aiops-labs ~]#
```

6. View all the deployments in your namespace.

```
kubectl get deploy -n ns-<userID>
```

The output of this command will be a row with the deployment name and ready column set to 1/1. The age column determines the duration of the deployment created.

7. View all the pods in your namespace.

```
kubectl get pods -n ns-<userID>
```

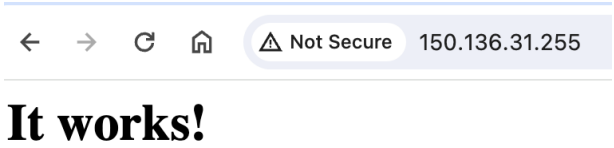
```
[root@aiops-labs ~]# kubectl get deploy -n ns-user16
NAME            READY   UP-TO-DATE   AVAILABLE   AGE
deploy-user16   1/1     1            1           81s
[root@aiops-labs ~]# kubectl get pods -n ns-user16
NAME                              READY   STATUS    RESTARTS   AGE
deploy-user16-6dbcf79894-snkzj    1/1     Running   0          105s
[root@aiops-labs ~]#
```

8. View all the services in your namespace.

```
kubectl get svc -n ns-<userID>
```

The output of this command is a row with service name and type set to Load Balancer. It shows you the details of CLUSTER-IP and EXTERNAL-IP.

9. Copy the IP address listed under the EXTERNAL-IP column and paste it in a browser to access your httpd application that is deployed within OKE cluster. The webpage will display:



10. Check the number of instances of pods running in your deployment.

```
kubectl get replicaset -n ns-<userID>
```

11. Scale up the current replicas by three so that Kubernetes can start new pods to scale up your service.

```
kubectl scale --replicas=3 deployment/deploy-<userID> -n ns-<userID>
```

On successful execution, output will be as given in following screen shot.

```
[root@aiops-labs ~]# kubectl scale --replicas=3 deployment/deploy-user16 -n ns-user16
deployment.apps/deploy-user16 scaled
[root@aiops-labs ~]# kubectl get pods -n ns-user16
NAME                              READY   STATUS    RESTARTS   AGE
deploy-user16-6dbcf79894-jn8wj    1/1     Running   0          8s
deploy-user16-6dbcf79894-q48f6    1/1     Running   0          8s
deploy-user16-6dbcf79894-snkzj    1/1     Running   0          4m7s
[root@aiops-labs ~]#
```

12. Check if you have three replicas running.

```
kubectl get replicaset -n ns-<userID>
```

13. View all the resources running in your namespace.

```
kubectl get all -n ns-<userID>
```

This command shows you all the pods, services, deployments, and replicaset running in your namespace within the OKE cluster.

```
[root@aiops-labs ~]# kubectl get all -n ns-user16
NAME                                READY   STATUS    RESTARTS   AGE
pod/deploy-user16-6dbcf79894-jn8wj   1/1     Running   0          62s
pod/deploy-user16-6dbcf79894-q48f6   1/1     Running   0          62s
pod/deploy-user16-6dbcf79894-snkzj   1/1     Running   0          5m1s

NAME                 TYPE           CLUSTER-IP      EXTERNAL-IP      PORT(S)        AGE
service/svc-user16   LoadBalancer   10.96.211.234   141.148.86.176   80:30680/TCP   4m18s

NAME                            READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/deploy-user16   3/3     3            3           5m1s

NAME                                       DESIRED   CURRENT   READY   AGE
replicaset.apps/deploy-user16-6dbcf79894   3         3         3       5m1s
[root@aiops-labs ~]#
```

14. View the pod logs. The `kubectl logs` command lets you inspect the logs for a particular pod.

```
kubectl logs <podname> -n ns-<userID>
```

15. Delete your deployment.

```
kubectl delete deploy deploy-<userID> -n ns-<userID>
```

16. Delete your service object.

```
kubectl delete svc svc-<userID> -n ns-<userID>
```

17. Run the following command and you'll not find any resources in your namespace.

```
kubectl get all -n ns-<userID>
```

18. Because all the resources are deleted, if you go back to your browser and hit refresh on the IP address you pasted earlier, the page will no longer respond.

Sequence 5. Using Secrets and a Deployment Manifest

To enable Kubernetes to pull an image from OCIR repository when deploying an application, you need to create a Kubernetes secret. The secret contains all the login details you would provide while logging in to OCIR using the `docker login` command, including your auth token.

1. Open the terminal of your Jupyter Lab
2. Run the following command to create a secret:

```
kubectl create secret docker-registry ocir-secret-<userID> \
--docker-server=phx.ocir.io \
--docker-username='ocuocictrng24/<userID>@acme.com' \
--docker-password='R5kwpS-xxxxx((]51r]]' \
--docker-email='<userID>@acme.com' -n ns-<userID>
```

Where,

- *<name-of-secret>-<userID>*: A unique name for the secret.
- `ocir.io` is the Oracle Cloud Infrastructure Registry name.
- *<tenancy-namespace>* is the Object Storage namespace string of the tenancy.
- `<oci-username>` provide the full username as given in **the profile menu**
- *<oci-auth-token>*: Use the auth token (random string) created earlier.

Important: If there are special character that shell may parse, then Create an Environment variable first and then substitute that variable in the following command. The command given below resulted in error in pulling image.

Example

```
kubectl create secret docker-registry ocir-secret-user16 \
--docker-server=phx.ocir.io \
--docker-username='ocuocictrng24/98631166-lab.user16' \
--docker-password=':f1:}h:xgpUD2HcNo>t;' \
--docker-email='user16@acme.com' -n ns-user16
```

The Solution :

```
DOCKER_PASS=':f1:}h:xgpUD2HcNo>t;'

kubectl create secret docker-registry ocir-secret-user16 \
  --docker-server=iad.ocir.io \
  --docker-username='ocuocictrng24/98631166-lab.user16' \
  --docker-password="$DOCKER_PASS" \
  --docker-email='user16@acme.com' \
  -n ns-user16
```

3. Run the following command to verify if the secret has been created:

```
kubectl get secrets -n ns-<userID>
```

```
[root@aiops-labs ~]# kubectl create secret docker-registry ocir-secret-user16 \
--docker-server=phx.ocir.io \
--docker-username='ocuocictrng24/98631166-lab.user16' \
--docker-password=':f1:}h:xgpUD2HcNo>t;' \
--docker-email='user16@acme.com' -n ns-user16
secret/ocir-secret-user16 created
[root@aiops-labs ~]# kubectl get secrets -n ns-user16
NAME                 TYPE                             DATA   AGE
ocir-secret-user16   kubernetes.io/dockerconfigjson   1      5s
[root@aiops-labs ~]#
```

Add the Secret and the Image Path to the Deployment Manifest.

1. Change to **IAD-DOP-LAB05-1-CR-01** directory and edit the file `HelloWorld-lb.yaml`

```
cd ~/IAD-DOP-LAB05-1-CR-01/
vi HelloWorld-lb.yaml
```

1) name: helloworld-deployment-*<userID>* ***(Line No 4)***
2) namespace: ns-*<userID>* ***(Line No 5)***
3) image: `<region-key>`.ocir.io/*<tenancy-namespace>*/*<repo-name>*:*<tag>* ***(Line No 19. Refer to Lab 4 Sequence 4 Step 1)***
4) replace `<secret-name>` with *ocir-secret-<userID>* ***(Line No 24)***

```
 1 apiVersion: apps/v1
 2 kind: Deployment
 3 metadata:
 4   name: helloworld-deployment-user16
 5   namespace: ns-user16
 6 spec:
 7   selector:
 8     matchLabels:
 9       app: helloworld
10   replicas: 1
11   template:
12     metadata:
13       labels:
14         app: helloworld
15     spec:
16       containers:
17       - name: helloworld
18     # enter the path to your image, be sure to include the correct region prefix
19         image: iad.ocir.io/ocuocictrng24/iad-aiops-lab04-ocir-1/webapp_user16:latest
20         ports:
21         - containerPort: 80
22       imagePullSecrets:
23     # enter the name of the secret you created
24       - name: ocir-secret-user16
25 ---
26 apiVersion: v1
27 kind: Service
28 metadata:
29   name: helloworld-service-user16
30   namespace: ns-user16
31   annotations:
32     oci.oraclecloud.com/load-balancer-type: "lb"
33     service.beta.kubernetes.io/oci-load-balancer-shape: "flexible"
34     service.beta.kubernetes.io/oci-load-balancer-shape-flex-min: "10"
35     service.beta.kubernetes.io/oci-load-balancer-shape-flex-max: "100"
36 spec:
37   type: LoadBalancer
38   ports:
39   - port: 80
40     protocol: TCP
41     targetPort: 80
42   selector:
43     app: helloworld
~
```

2. Also, replace the placeholders in the Service section:

   5) name: helloworld-service-*<userID>* ***(Line No 29)***
   6) namespace: ns-*<userID>* ***(Line No 30)***

Sequence 6. Deploy the Sample Web Application to OKE Cluster and Verify

After making changes to manifest, you are ready to deploy the application to the OKE cluster.

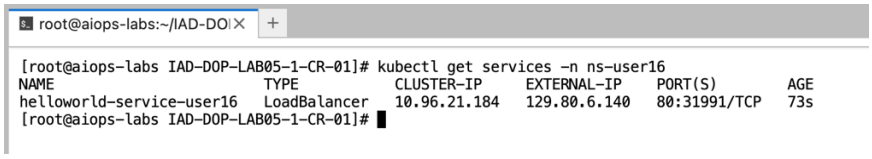
1. Run the following command:

   ```
   kubectl apply -f HelloWorld-lb.yaml
   ```

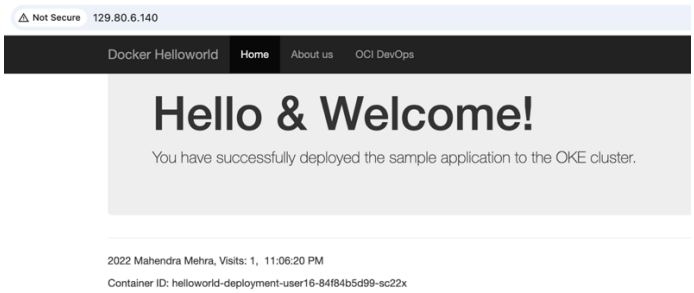
   The HelloWorld Service Load Balancer is implemented as an OCI Load Balancer with a backend set to route incoming traffic to the cluster nodes.

2. Now run the command:

   ```
   kubectl get services -n ns-<userID>
   ```



3. Launch an Internet Browser and enter the HelloWorld-Service Load Balancer's External/Public IP into the address bar to access the deployed application. The load balancer routes the request to available nodes in the cluster.



   Now comes the fun part! Let's pretend your sample web application has suddenly gained popularity and you are now required to allocate more resources to it. The OKE cluster is running on a single node pool with three worker nodes, thus you can easily scale your deployment.

   a. To scale up twice as much and run an additional pod for your current single pod deployment, run the command:

```
kubectl -n ns-<userID> scale --replicas=2 deployment/helloworld-deployment-<userID>
```

You will see a confirmation for deployment scaling on screen.

b. Further, to see pod and deployment details, run the command:

```
kubectl get all -n ns-<userID>
```

Sequence 7. Clean Up the Resources Deployed Within OKE Cluster

Clean up the resources deployed within OKE cluster.

1. To delete the sample web application and all other resources you created on the cluster, run the following command:

```
kubectl delete -f HelloWorld-lb.yaml -n ns-<userID>
```

2. To confirm the resources are cleared, run the command:

```
kubectl get all -n ns-<userID>
```

You will observe that no resources are found in the namespace.

Important Note: Do not delete the namespace and entry created in the kubeconfig file in this lab, because they will be required in the upcoming labs.

Congratulations! You have successfully deployed a sample web application to the OKE cluster using secrets and deployment manifest. Purge Instructions